

beginner roadmap · 6 steps · zero to first real build

How to Learn Electronics From Zero

The order I'd do it in if I started again: build first, read second, solder early, measure everything, then make one thing you actually want. No degree required, about \$60 of parts, and you'll have something blinking on day one.

By Enes · @1nska · Electrical Engineering student – a Build List quick guide

“you don't learn electronics by reading about electrons. you learn it by making an LED do what you told it to.”

what's inside: the six steps · a \$60 bench · the first five builds · the only theory you need · three decisions explained

the six steps

STEP 01

Skip the theory at first. Get a breadboard and an Arduino kit.

DO THIS

Buy a starter kit, not individual parts. A \$30-40 Arduino Uno clone kit gives you the board, a breadboard, jumper wires, a resistor assortment, LEDs, buttons, a buzzer, a potentiometer and a few sensors – everything for the first month.

THE TRAP

Buying a 'proper' bench setup before you know what you'll use. A \$200 oscilloscope on day one teaches you nothing; a \$2 LED does.

YOU'RE DONE WHEN

The kit is on your desk and the Arduino IDE (or PlatformIO) is installed and sees the board.

STEP 02

Blink an LED. Then a buzzer. Then a button.

DO THIS

Upload the Blink example, then change the delay from 1000 ms to 100 ms and watch it change. Wire an LED with a 220 ohm resistor on pin 13. Add the buzzer with `tone()`. Add a button with `pinMode(INPUT_PULLUP)` so the LED only lights while it's pressed.

THE TRAP

Copy-pasting a whole project on day two. Every build here is under 15 lines – type them, break them, fix them.

YOU'RE DONE WHEN

You can make the LED, the buzzer and the button work together without looking anything up.

STEP 03

Now learn the theory you just used.

DO THIS

You've already used Ohm's law (that 220 ohm resistor), Kirchhoff's laws (the current through your LED loop) and a pull-up resistor. Go back and read exactly those: $V = IR$, series vs parallel, voltage dividers, why the LED needs a resistor at all. The concepts stick because you've held the parts.

THE TRAP

Trying to finish a 900-page textbook before touching a wire. Read one chapter, build one thing, repeat.

YOU'RE DONE WHEN

You can calculate the resistor for any LED from its datasheet and explain why 5 V straight into an LED kills it.

STEP 04

Learn to solder early. It's easier than it looks.

DO THIS

A \$20 temperature-controlled iron, 0.8 mm leaded or lead-free solder, flux and a practice kit (a \$5 through-hole 'skill kit'). Tin the tip, heat the pad and the leg together for two seconds, feed solder into the joint, not the iron. Twenty joints and you'll have it.

THE TRAP

Waiting until a project 'needs' soldering. Breadboards fall apart the moment you move them – soldering is what makes something yours to keep.

YOU'RE DONE WHEN

Your joints look like small shiny cones, not grey blobs, and a soldered LED circuit works first try.

STEP 05

Measure everything. Multimeter first, oscilloscope later.

DO THIS

A \$15 multimeter answers 90% of beginner questions: is there voltage here, is this wire actually connected (continuity beep), how much current does this draw. Measure your LED circuit and compare with what Ohm's law predicted. Only reach for a scope when you start caring about signals over time (PWM, serial, audio).

THE TRAP

Guessing. 'It doesn't work' is not a diagnosis. 'Pin 13 is at 4.9 V but the LED anode is at 0 V' is – and that takes ten seconds with a meter.

YOU'RE DONE WHEN

You debug by measuring before you debug by re-wiring.

STEP 06

Build one real project you actually want to use.

DO THIS

Not a tutorial – something with a job: a plant-moisture alarm, a desk lamp that dims with a knob, a door sensor that texts you, a temperature logger for your room. Pick one, make a parts list, build it on a breadboard, then solder it onto perfboard and put it in a box.

THE TRAP

Ten half-finished projects. One finished thing you use every day teaches more than ten demos, and it's the one you'll show people.

YOU'RE DONE WHEN

It's been running on your desk for a week and you've fixed at least one thing that broke.

the \$60 starter bench

- Arduino Uno clone starter kit (board, breadboard, wires, LEDs, resistors, buttons, buzzer, pot, sensors) – \$30-40
- Digital multimeter with continuity beep – \$15
- Temperature-controlled soldering iron kit + solder + flux – \$20-25
- Through-hole soldering practice kit – \$5
- USB cable and a phone charger as your 5 V supply – \$0
- Later, not now: ESP32 board (\$6) once you want Wi-Fi, a basic oscilloscope (\$60+) once you care about signals

the first five builds, in order

- Blink – change the timing, then blink two LEDs alternately
- Button-controlled LED – learn INPUT_PULLUP and why the button reads backwards
- Buzzer melody – tone() and a short array of notes
- Potentiometer dimmer – analogRead in, PWM out with analogWrite
- Light-sensor night light – a photoresistor in a voltage divider turns the LED on in the dark

the only theory you need for month one

- Ohm's law, $V = IR$: picks every resistor you'll ever add.
- Kirchhoff's current and voltage laws: what goes in comes out; voltages around a loop add to zero. That's how you reason about any circuit.
- Series vs parallel: series shares current, parallel shares voltage.
- Voltage divider: two resistors turn any sensor into a readable voltage.
- LED current limiting: $(5\text{ V} - 2\text{ V}) / 220\text{ ohm}$ is about 14 mA – bright and safe.
- Pull-up and pull-down resistors: why a floating input reads random noise.
- PWM: switching fast enough that a dimmer looks smooth to your eye.

why this order

→ Why build before reading?

theory read without a circuit in front of you evaporates in a week; theory read to explain something you just watched happen sticks for years. The kit costs less than the textbook.

→ Why an Arduino instead of a bare microcontroller?

the Uno hides nothing that matters to a beginner – you still wire every resistor and read every pin – but it removes the toolchain and programmer setup that stops most people before the first blink. Move to bare chips or an ESP32 when a project needs it.

→ Why a multimeter before an oscilloscope?

a meter tells you what's true right now, and almost every beginner bug is a wrong voltage or an open wire. A scope shows how a signal changes over time, which you won't need until you're debugging PWM, serial or audio.

That's the whole roadmap.

Six steps, one kit, one project you keep. Everything past this – motors, radios, PCBs, power supplies – is the same loop: build the small version, then read exactly the theory you just used, then measure it.

Comment LEARN on the reel and I'll send you this guide. Follow for the next one – the first five builds, step by step.

– Enes (@1nska) · Electrical Engineering student